CSI 4106 Introduction to Artificial Intelligence

Assignment 3: Alien Pet Health, Deep Learning

Marcel Turcotte

Version: Oct 26, 2025 11:09

◎ Learning Objectives

- **Design** and **train** fully connected neural networks using Keras to solve a binary classification task, while **interpreting** the role of layers, activation functions, and learning dynamics.
- **Diagnose** underfitting and overfitting through visualization of learning curves and quantitative performance metrics.
- **Apply** and **compare** regularization techniques such as L2 penalties, dropout, and early stopping to improve model generalization.

Deep neural networks excel at discovering structure in large, unstructured datasets such as images, text, speech, or sensor sequences, where explicit feature engineering is difficult.

- Data is large (tens of thousands to hundreds of millions).
- Features are raw and high-dimensional (hundreds to millions)
- Relationships are complex or compositional (hierarchies, temporal dependencies).
- Representation learning can replace or outperform manual feature design.

I considered other choices of the dataset for this assignment. However, to streamline the assignment and minimize training time, which could potentially extend to several hours,[1] I decided to retain the dataset used in Assignments 1 and 2. This choice allows us to achieve the same learning objectives with less demand on your time. The dataset was specifically designed to be conducive to deep learning applications, and I am pleased to report that it has proven effective. In my experiments, the top-performing deep learning models achieved a validation F1 score in the low nineties.

[1] Training the model described in the image_classification_from_scratch.ipynb notebook on a laptop with an M2 Max processor, comprising 8 performance cores, 4 efficiency cores, and 64 GB of RAM, took over 7 hours as of October 25, 2025. This extended training duration is not ideal for experimental iterations.

Submission

- **Deadline:**
  - Submit your notebook by November 10, 11 PM.
- **Individual Assignment:**
  - This assignment must be completed individually.
- **Submission Platform:**
  - Upload your submission to Brightspace under the Assignment section (Assignment 3).
- **Submission Format:**
  - One notebook (`.ipynb`) with brief narrative + code + plots.

Important Notice: Ensure that your notebook executes without errors in Google Colab, which will be our standard reference environment. While you may choose to develop your notebook on a different platform, it is critical that your teaching assistant can run it in Colab without needing to install additional librairies or transfer data manually. Retain all execution results, including plots, within the notebook so that your teaching assistant can evaluate your work without needing to execute any cells.

Requirements

1. Data

We utilize the dataset from the prior assignment, which encompasses features deemed relevant for predicting health outcomes in companion species. This comprehensive dataset is compiled from multiple colonies and clinics. Each record includes data from intake forms and sensor readings, aggregated from various clinics, vendors, and time zones. All these diverse data sources have been integrated into a single CSV file.

For this assignment, we use a dataset in which all tasks from the prior assignment have been thoroughly completed. Notably, Assignment 3 uses the dataset alien_pet_health-realism-clean.csv, from assignment Assignment 2.

The Jupyter Notebook template includes source code designed for data loading and preprocessing.

2. Tasks

The tasks are arranged in a specific order to optimize brevity and clarity. Although you have some flexibility in organizing your code, it is important that your teaching assistant can effortlessly identify the code segments related to each task.

Suggestions: Start with a small model (e.g., 8 units) and train for 10–20 epochs using default hyperparameters. Expand exploration only after observing reasonable learning curves and metrics. Create reusable helper functions and consistently plot training and validation loss curves. Log experiments concisely, plotting loss curves only for the best model for each task.

(1) **Build a Simple Feed-Forward Network**

- One hidden layer (8 units, ReLU), output = sigmoid.
- Use binary cross-entropy and the Adam optimizer.
- Plot loss and F1 score vs. epochs (train + validation) for 100 epochs.
- Report precision, recall, F1, and ROC/AUC for the validation set.
- Does the model underfit or overfit? How can you tell?

(2) **Overfitting**

A common and effective strategy when developing a deep learning model is to gradually increase model complexity (more layers, more units, richer activation functions) until overfitting becomes visible.

Once overfitting is detected, regularization techniques such as L2 weight penalties, dropout, and early stopping are introduced to improve generalization.

- Construct a model intentionally designed to overfit the training dataset.
- Visualize the loss and F1 score over epochs for both training and validation datasets.
- Report precision, recall, F1, and ROC/AUC.
- Describe the approach used to achieve overfitting in the model.
- Explain the evidence that indicates the model is overfitting.

(3) **Eearly Stopping**

Integrate an early stopping mechanism into the training. This involves stopping the training process and reverting to the model's best-performing weights if there is no observed improvement for a duration of 10 consecutive epochs.

- Plot loss and F1 score vs. epochs (train + validation).
- Report precision, recall, F1, and ROC/AUC.

(4) **Architecture Exploration**

To streamline the assignment, test various activation functions using a restricted set of configurations. Begin with a minimal setup and iteratively refine it based on results. Implement early stopping, as previously described, to effectively decrease training time.

Experiment with the following hyperparameters and test at least 12 architectures.

- Number of hidden layers = 1, 2, 3
- Units = 8, 16, 32, 64
- Activation = ReLU, tanh

For each configuration tested, record precision, recall, F1, and ROC/AUC.

For your best configuration, plot loss and F1 score vs. epochs (train + validation).

Does the model underfit or overfit? How can you tell?

Write your observations.

(5) **Regularization**

Select an architecture demonstrating overfitting, which you will use for both experiments below.

To facilitate analysis and visualization, avoid early stopping and train the models for at least 100 epochs.

1. **Apply L2 penalty:** $\lambda \in \{0.001, 0.0001\}$.
    - For each configuration tested, record precision, recall, F1, and ROC/AUC.
    - Visualize the behaviour of the best configuration by plotting the loss and F1 score against epochs for both training and validation datasets, comparing models with and without regularization.
2. **Apply Dropout:** rate $\in \{0.25, 0.5\}$
    - For each configuration tested, record precision, recall, F1, and ROC/AUC.
    - Visualize the behaviour of the best configuration by plotting the loss and F1 score against epochs for both training and validation datasets, comparing models with and without regularization.

Analyze the influence of each regularization techniques on the overall model performance.

Does it take fewer or more iterations before you can observe overfitting?

Which technique was more effective at mitigating overfitting?

Record all your observations.

(6) **Model Evaluation**

Select the best architecture from the experiments discussed above, including regularization if applicable, and provide a rationale for your choice.

- Apply the best model to the test set.
- Report precision, recall, F1, and ROC/AUC.
- Does the performance of your best deep learning model represent a substantial or meaningful improvement over the results obtained in Assignment 2?
- Record all your observations.

Bonus (Optional)

You can obtain up to 2 bonus marks if you can further improve the test performance of your model.

Suggestions:

- Perform a more thorough hyperparameter exploration, potentially combining regularization techniques.
- Develop an ensemble classifier by training three distinct deep learning models, each initialized with a different random seed. Subsequently, compute the average of their predicted probabilities or take a majority vote.

3. Documentation of Exploratory Analysis

Your report should comprehensively document the entire process followed during this assignment. The Jupyter Notebook must include the following:

- Your name, student number, and a report title.
- A section for each step of the exploratory analysis. Each section should contain the relevant Python code and explanations or results.
  - For sections requiring Python code, include the code in a cell.
  - For sections requiring explanations or results, include these in a separate cell or in combination with code cells.
- Ensure logical separation of code into different cells. For example, the definition of a function should be in one cell and its execution in another. Avoid placing too much code in a single cell to maintain clarity and readability.
- The notebook you submit must include the results of the execution, complete with graphics. In other words, your teaching assistant should be able to grade your notebook without needing to execute the code.

Evaluation (48 points)

- **Overall Effort in the Report (3)**
  - The writing should be clear and descriptive, allowing readers to readily comprehend the actions taken, the methods employed, and the rationale behind them.
  - Distinguish clearly between the narrative text, code, and results to enhance readability.
  - Use tables and/or graphs to visually compare different methodologies effectively.
  - Ensure the report contains sufficient detail to enable the reproducibility of the findings.
- **Code Quality (3)**
  - Clear structure, descriptive names, short functions.
  - One command re-creates results, random seeds fixed, output paths managed, no need for manual intervention.
  - Dependencies clearly documented.
  - Comprehensive validations and graceful error handling.
- **Exploratory Analysis (40)**
  1. Build a Simple Feed-Forward Network (10)
  2. Overfitting (6)
  3. Eearly Stopping (6)
  4. Architecture Exploration (8)
  5. Regularization (6)
  6. Model Evaluation (4)
- **Resources and References (2)**
  - Cite any part of your code that is sourced from websites, including tutorial sites or Stack Overflow.
  - Reference any theory or algorithms utilized that are found in books, slides, or tutorials.

Resources

As previously stated, it is important to properly attribute any portions of your code that are sourced from external materials, such as websites, textbooks, or other resources.

In contemporary software development, artificial intelligence (AI) is increasingly utilized by programmers to boost productivity, a trend expected to persist. To remain competitive in the

job market, it is advantageous to integrate these technologies into your skill set. Nonetheless, it is crucial to thoroughly comprehend the underlying concepts on which evaluations are based, given that AI tools may not be accessible during in-person assessments.

If you use AI assistance, it is essential to meticulously document all interactions. Your report should specify the tools and their versions, accompanied by a complete transcript of these interactions. Most AI platforms automatically log conversations, so it is advisable to initiate a new conversation specifically for the assignment and consistently use this thread throughout your work. Ensure that this conversation remains focused exclusively on the assignment. Include the transcript of this conversation in the reference section of your Jupyter Notebook.

Questions

- Please direct any questions you have to the Assignment topic within the discussion forum on Brightspace. This approach is highly encouraged, as it facilitates collective learning by enabling all students to access both the questions and the responses provided by the teaching assistants.
- If necessary, you may also email one of the four teaching assistants directly, although this method is less preferred.